APG: Agentic Static-Analysis Infrastructure on LLVM-IR Graph Databases

Zexin Zhong
University of Technology Sydney
Sydney, Australia
zexin.zhong@student.uts.edu.au

Yanjun Zhang
Griffith University
Queensland, Australia
yanjun.zhang@griffith.edu.au

Yulei Sui
University of New South Wales
Sydney, Australia
y.sui@unsw.edu.au

Xiao Cheng
Macquarie University
Sydney, Australia
xiao.cheng@mq.edu.au

arXiv:2611.02349v1 [cs.SE] 2 Nov 2026

***Abstract*—LLM-based coding agents increasingly rely on external program knowledge, yet access to static-analysis graphs alone does not ensure appropriate operation selection or evidence interpretation. We present APG, an infrastructure for semantics-grounded static program analysis. Built on SVF and LLVM, it decomposes compiler-derived program analyses into atomic capabilities that LLM agents can select, compose, and orchestrate for goal-directed, cross-graph investigation, with query safeguards and evidence traces preserving inspectable results and provenance. APG provides 38 atomic capabilities. On analysis requests arising during benchmark repair runs, 35 achieve first-attempt query-generation success rates above 50%, including 13 at 100%. Evaluation on 812 C/C++ defects shows higher repair success than the evaluated baselines. Compared with Passive Graph Push, APG improves first-iteration repair success by 17.6 and 18.0 percentage points on the two benchmark collections and reduces average LLM input-token consumption by 90.8%. Our tool is available at https://github.com/JasonZhongZexin/APG-ICSE-Demo and its demonstration video at https://youtu.be/vkuExZUjGnw.**

I. Introduction

Interprocedural static analysis on LLVM IR, as realized by frameworks such as SVF [1], computes precise structural facts about system-level C/C++ programs, including pointer aliasing, value-flow dependencies, and control-flow reachability across compilation units. Such structural evidence can inform fault localization [2] and program repair [3], yet providing agents with usable access to these analyses remains a challenge. RLCE retrieves repository context for repair [4]; STALL+ integrates static analysis into code completion [5]; and RepoGraph provides graph-based repository navigation [6]. These systems focus on repository context, code completion, and navigation. Exposing LLVM-IR interprocedural analyses as reusable, agent-invocable capabilities poses a further infrastructure challenge.

These facts should instead be supplied to agents directly, letting the model focus on higher-level reasoning such as defect interpretation, root-cause localization, and fix synthesis. A graph database is a natural vehicle: many program analyses reduce to graph reachability [7], and a declarative language such as Cypher lets an agent express *what* structural information it needs without encoding *how* to traverse the analyzer's internal data structures, a separation far more amenable to LLM interaction than imperative analysis APIs.

However, *merely connecting an LLM to a static-analysis database does not make it capable of using static analysis correctly*. Program-analysis graphs encode specialized relationships whose meanings depend on their underlying analysis semantics. An agent must determine which analysis operation is appropriate, how to express its intended goal, and what conclusions the returned evidence supports. A schema-valid query may still investigate the wrong relationship, while an empty result does not necessarily establish the absence of a program property.

This gap motivates two design principles. First, program-analysis infrastructure should be *agent-facing*: designed for LLM agents as primary consumers, exposing semantically meaningful analysis capabilities rather than only low-level graph operations. Second, it should be *inspectable*: agent-generated operations should be checked against supported constraints, and the evidence underlying analysis conclusions should remain traceable.

We present APG, an infrastructure for semantics-grounded static program analysis. Built on SVF and LLVM, APG decomposes compiler-derived program analyses into atomic, agent-invocable capabilities through a semantics-aware interface. LLM agents select and orchestrate these capabilities for goal-directed, iterative investigation of program structures, while query safeguards and provenance-aware evidence organization support constrained and inspectable program investigation.

The main tool contributions are:

- **LLVM-IR graph database infrastructure.** APG materializes five SVF-derived interprocedural analysis graphs (PAG, ICFG, Call Graph, SVFBasicBlock Graph, and CHG) into a queryable graph database, making compiler-precise program relationships accessible via declarative Cypher queries.
- **Agent-facing analysis interaction and orchestration.** APG decomposes static analyses into atomic capabilities that LLM agents select and orchestrate through a semantics-aware interface, enabling goal-directed, iterative investigation of program structures.
- **Inspectable, evidence-grounded investigation.** APG integrates query safeguards and provenance-aware evidence organization to make analysis operations and their supporting structural evidence inspectable.

II. Tool Design and Implementation

Fig. 1 illustrates the architecture of APG. Building on SVF [1], the framework materializes five compiler-derived program-analysis graphs (PAG, ICFG, Call Graph, SVFBasicBlock Graph, and CHG) in TuGraph [8] for on-demand structural queries.

Given a defect description or analysis question, the agent formulates analysis goals and selects semantic capabilities through a semantics-aware interface. An iterative controller coordinates query assessment, execution, and follow-up investigation. Retrieved results are consolidated into local and global evidence summaries, while analysis traces preserve the operations, queries, and execution feedback for inspection. The accumulated evidence supports fault diagnosis and downstream automated repair.

The architecture realizes two principles: *agent-facing interaction*, which exposes static-analysis operations as agent-invocable semantic capabilities, and *inspectable investigation*, which constrains operations and retains traceable evidence. The following subsections describe the three mechanisms.

A. Semantics-Grounded Analysis Capabilities

Rather than exposing program graphs solely as schemas, the semantics-aware interface interposes a semantic layer between an agent's analysis intention and its executable graph operation. It comprises a Graph Semantics Catalog, an Atomic Capability Catalog, and reusable graph-query operators.

The design separates three levels: *analysis intent* (the structural information being sought), *semantic capability* (the corresponding program-analysis operation), and *query realization* (expressing that operation over the graph). The Graph Semantics Catalog describes graph entities, relationships, and their analysis meanings. Atomic capabilities provide reusable abstractions for selecting meaningful operations.

This separation is necessary because schema validity alone does not establish analysis correctness. For example, determining whether function f can transitively call g requires a reachability traversal, not a direct-edge lookup. The semantic layer thus guides both operation selection and evidence interpretation.

Each capability is exposed as an invocable interface. Given the selected capability and the current analysis intent, the LLM generates Cypher using the associated query templates and reusable Cypher operators. Query Guard checks syntax, safety constraints, and compatibility with the query features supported by the deployed TuGraph engine. Schema Guard checks whether the node and edge properties referenced by the query are defined in the graph schema. Goal/query alignment is assessed separately during investigation.

B. Goal-Directed Structural Investigation

APG organizes exploration as a sequence of explicit analysis goals and evidence-acquisition steps. An `AttemptController` coordinates this process under bounded execution budgets, so the agent progressively investigates program relationships rather than retrieving a fixed collection of graph context.

At each step, the agent identifies an information need, selects a capability, and generates a candidate Cypher query. Before execution, the query undergoes goal/query alignment assessment, Query Guard, and Schema Guard. Alignment assessment examines whether the query expresses the intended analysis goal, beyond syntax and schema validity. Only queries passing all applicable checks proceed to execution; otherwise, the agent revises the query or replans the investigation.

Returned results are interpreted against the current goal. A zero-hit result establishes only that no matching evidence was found within the queried scope. It may reflect an absent relationship, incomplete graph information, or an undetected query-goal mismatch.

Based on accumulated evidence, the agent determines whether further investigation is needed. For example, diagnosing a cross-file memory defect may require combining call-graph, pointer, and control-flow evidence from different graphs. Each retrieval step is thus tied to an explicit analysis objective, unlike passive graph-context injection.

C. Inspectable Evidence-Grounded Reasoning

APG separates retrieved program evidence from the conclusions the LLM derives from it. Individual query results are processed into local summaries of relevant entities and structural relationships. A global summary consolidates evidence across investigation steps, providing structured context for subsequent reasoning without requiring the full collection of raw query outputs.

Crucially, summarization does not replace the underlying records. APG maintains analysis traces and provenance linking operations, executed queries, graph results, and execution feedback. Users can therefore inspect the evidence available to the agent and examine potential failures in the investigation process. The same investigation mechanism supports multiple downstream tasks: structural program analysis, fault localization and root-cause diagnosis, and automated repair (which extends the process with patch generation and test feedback). The resulting *white-box* property concerns the inspectability of external analysis operations and their supporting evidence, not visibility into the LLM's internal computations or formal correctness guarantees.

III. Evaluation

We evaluate APG on **RQ1: capability coverage and query generation**, **RQ2: query reliability**, and **RQ3: repair effectiveness and retrieval efficiency**.

A. Experimental Setup

We use 812 C/C++ defects: 277 from Multi-SWE-bench [9] and 535 from Defects4C [10] and ManyBugs [11]. Baselines are Text-RAG (source retrieval), AST-Agent (syntax-level context), and Passive Graph Push (preassembled graph context). All configurations use GPT-4o with temperature 0.0

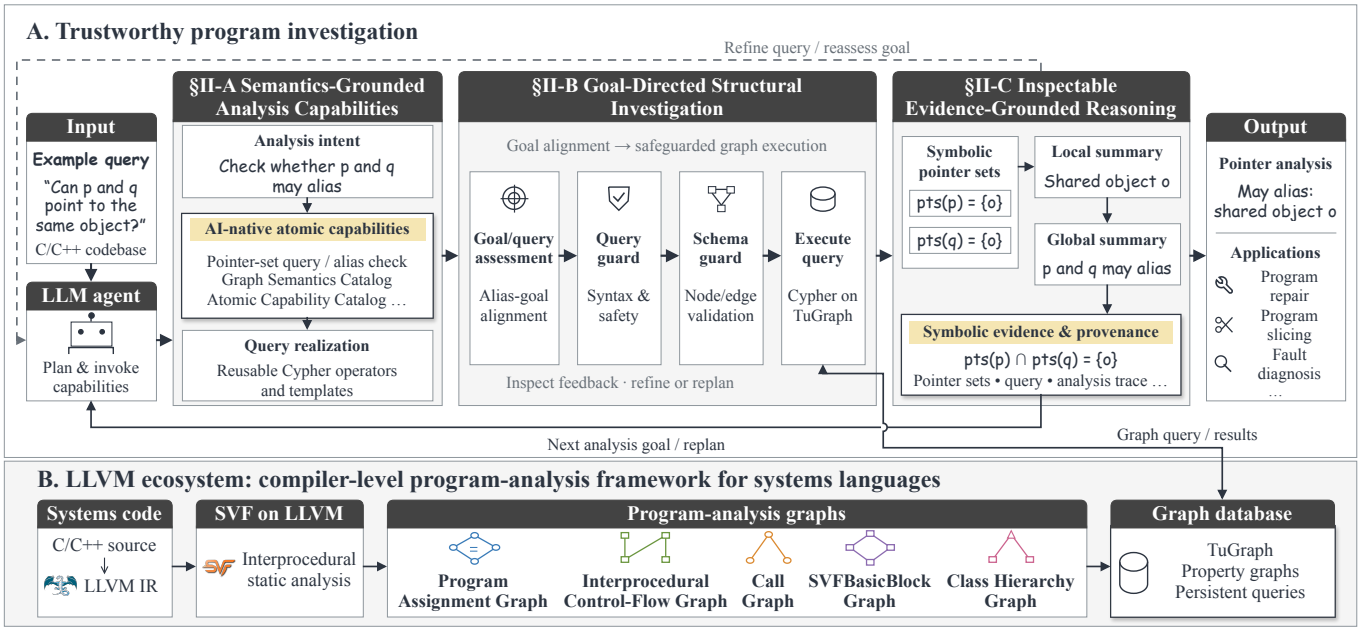

Fig. 1: Overview of APG.

TABLE I: Capability coverage and capability-level Query S@1. Interval columns report numbers of capabilities.

		Query S@1			
Category	**Count**	**100%**	**(50, 100)%**	**≤50%**	**Representative capabilities**
CallGraph	8	3	5	0	Call chains; reachability; self-recursion; mutual recursion
ICFG	8	2	5	1	Branch context; guard candidates; interprocedural paths; loop detection
PAG	10	4	5	1	Argument flow; load/store uses; parameter passing; approximate alias evidence
CHG/Type	5	0	4	1	Class hierarchy; virtual-dispatch candidates; override relations; type/field information
Cross-graph	7	4	3	0	Entity mapping; evidence aggregation; check-before-use; sanitizer-to-sink investigation
Total	**38**	**13**	**22**	**3**	Excludes five source/symbol capabilities.

TABLE II: Query reliability and repair success (%).

Configuration	**IQR ↓**	**Repair S@1 ↑**
Without Graph Semantics Catalog	68.4	4.8
Without Guards/Replanning	14.2	21.5
Full Pipeline	0.8	32.8

TABLE III: Repair success within one and five sequential iterations (%).

	Multi-SWE		**D4C/ManyBugs**	
Method	**S@1**	**S@5**	**S@1**	**S@5**
Text-RAG	3.9	8.2	2.4	5.8
AST-Agent	13.8	20.4	8.9	14.5
Passive Push	15.2	24.1	11.2	18.4
APG	32.8	45.4	29.2	41.6

and a clean checkout for each defect, allowing up to five sequential repair iterations. Each iteration includes context acquisition, patch generation, and compilation/test validation. Repair Success@k measures success within k sequential iterations, rather than independently sampled Pass@k.

B. RQ1: Capability Coverage and Query Generation

APG provides 38 atomic analysis capabilities across five graph categories (Table I). For each capability, *Query S@1* measures the proportion of analysis requests whose initial Cypher query correctly expresses the intended goal. Corrections and retries do not count as first-attempt successes.

Of the 38 capabilities, 35 achieve Query S@1 above 50%, including 13 at 100%. These rates characterize the benchmark workload, rather than guaranteeing the same outcomes for unseen requests. Capabilities compose through shared entities and source locations. For example, parameter propagation identifies a use location for branch-context retrieval, while call-chain exploration supplies its calling context.

C. RQ2: Query Reliability

We ablate the Graph Semantics Catalog and the combined guards/replanning mechanisms. Invalid Query Rate (IQR) measures the proportion of query observations containing Cypher syntax or graph-schema violations.

As shown in Table II, the full pipeline reduces IQR to 0.8% and achieves the highest repair S@1. The results support the role of the Graph Semantics Catalog and query safeguards in maintaining query validity.

D. RQ3: Repair Effectiveness and Retrieval Efficiency

APG achieves the highest S@1 and S@5 on both benchmarks (Table III). Compared with Passive Graph Push, S@1 improves by 17.6 and 18.0 percentage points on Multi-SWE-bench and Defects4C/ManyBugs. Under the same accounting scope, average LLM input tokens per defect drop from 45.8K (Passive Push) to 4.2K (APG), a 90.8% reduction. This is consistent with acquiring structural evidence on demand rather than injecting preassembled graph context.

IV. Related Work

Repository-Level Agent Interaction. SWE-agent enables LLM agents to navigate, edit, and test code repositories [12]. RepoCoder and RLCE improve repository-level context retrieval for code generation and repair [13], [4], while RepoGraph provides graph-based repository navigation [6]. These approaches facilitate repository exploration. APG addresses a complementary challenge: enabling agents to invoke program-analysis operations with explicit semantics, rather than relying solely on retrieved repository context.

Structural Knowledge Integration. STALL+ integrates static-analysis information into repository-level code completion [5], while CGM incorporates code graphs into an LLM's attention mechanism [14]. More closely related, CT-Repair enables agents to query static Code Property Graphs and dynamic Temporal Execution Graphs to develop evidence-grounded repair strategies [15]. Unlike model-integrated graph representations or repair-specific evidence workflows, APG emphasizes reusable semantic analysis capabilities exposed through an external, agent-facing interface.

Agentic Program-Analysis Querying. Declarative query languages support expressive program analysis [16]. QLCoder addresses LLM interaction with static analysis by synthesizing CodeQL vulnerability queries using syntax guidance and execution feedback [17]. APG targets a different abstraction: mapping analysis intentions to semantic capabilities and executable graph queries, then organizing multi-round investigation around inspectable structural evidence and provenance.

V. Conclusion

We presented APG, an infrastructure that materializes five SVF-derived interprocedural analysis graphs into a queryable graph database and decomposes the resulting static analyses into 38 atomic, agent-invocable capabilities. LLM agents select and orchestrate these capabilities through a semantics-aware interface for goal-directed, cross-graph program investigation, while query safeguards and provenance-aware evidence organization keep the investigation process constrained and inspectable.

References

[1] Y. Sui and J. Xue, "Svf: interprocedural static value-flow analysis in llvm," in *Proceedings of the 25th International Conference on Compiler Construction*, ser. CC '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 265–266. [Online]. Available: https://doi.org/10.1145/2892208.2892235

[2] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.

[3] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen, "A systematic literature review on large language models for automated program repair," 2024. [Online]. Available: https://arxiv.org/abs/2405.01466

[4] Y. Chen, J. Wu, X. Ling, C. Li, Z. Rui, T. Luo, and Y. Wu, "When large language models confront repository-level automatic program repair: How well they done?" in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, ser. ICSE-Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 459–471. [Online]. Available: https://doi.org/10.1145/3639478.3647633

[5] J. Liu, Y. Chen, M. Liu, X. Peng, and Y. Lou, "Stall+: Boosting llm-based repository-level code completion with static analysis," 2024. [Online]. Available: https://arxiv.org/abs/2406.10018

[6] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, "RepoGraph: Enhancing ai software engineering with repository-level code graph," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/4a4a3c197deac042461c677219efd36c-Abstract-Conference.html

[7] T. Reps, "Program analysis via graph reachability," *Information and Software Technology*, vol. 40, no. 11–12, pp. 701–726, 1998.

[8] Ant Group, "Tugraph: A high-performance distributed graph database," 2024, https://tugraph.tech/.

[9] D. Zan, Z. Huang, W. Liu, H. Chen, L. Zhang, S. Xin, L. Chen, Q. Liu, X. Zhong, A. Li, S. Liu, Y. Xiao, L. Chen, Y. Zhang, J. Su, T. Liu, R. Long, K. Shen, and L. Xiang, "Multi-swe-bench: A multilingual benchmark for issue resolving," 2025. [Online]. Available: https://arxiv.org/abs/2504.02605

[10] J. Wang, X. Xie, Q. Hu, S. Liu, J. Yu, J. Kong, and Y. Li, "Defects4c: Benchmarking large language model repair capability with c/c++ bugs," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 254–265.

[11] C. Le Goues, N. Holtschulte, E. K. Smith, Y. Brun, P. Devanbu, S. Forrest, and W. Weimer, "The ManyBugs and IntroClass benchmarks for automated repair of C programs," *IEEE Transactions on Software Engineering*, vol. 41, no. 12, pp. 1236–1256, 2015. [Online]. Available: https://doi.org/10.1109/TSE.2015.2454513

[12] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[13] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 2471–2484. [Online]. Available: https://aclanthology.org/2023.emnlp-main.151/

[14] H. Tao, Y. Zhang, Z. Tang, H. Peng, X. Zhu, B. Liu, Y. Yang, Z. Zhang, Z. Xu, H. Zhang, L. Zhu, R. Wang, H. Yu, J. Li, and P. Di, "Code graph model (CGM): A graph-integrated large language model for repository-level software engineering tasks," 2025. [Online]. Available: https://arxiv.org/abs/2505.16901

[15] Z. Huang, L. Xu, and H. Zhang, "Multi-perspective agentic program repair via code property graphs and temporal execution graphs," 2026. [Online]. Available: https://arxiv.org/abs/2607.12605

[16] P. Avgustinov, O. de Moor, M. P. Jones, and M. Schäfer, "QL: Object-oriented queries on relational data," in *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 56. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016, pp. 2:1–2:25. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2016.2

[17] C. Wang, Z. Li, S. Dutta, and M. Naik, "QLCoder: A query synthesizer for static analysis of security vulnerabilities," 2026. [Online]. Available: https://arxiv.org/abs/2511.08462